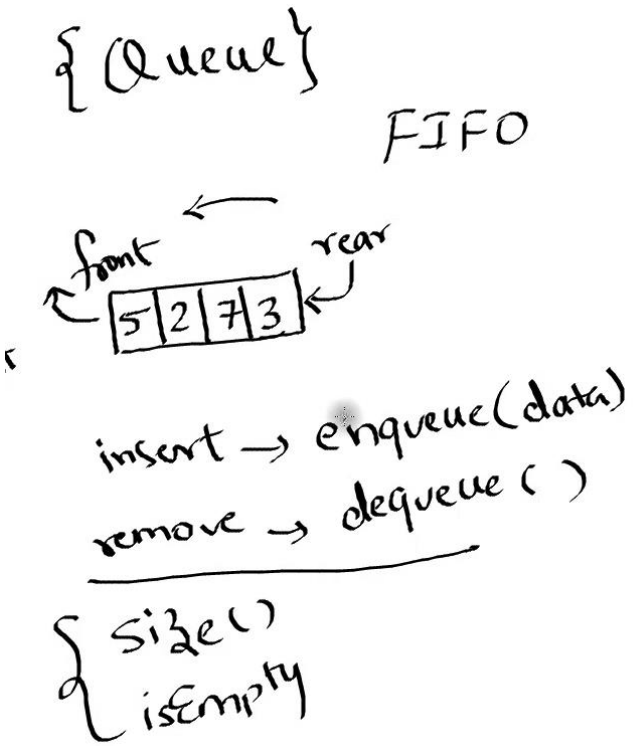
A **queue** is a linear, **First-In, First-Out (FIFO)** data structure where it have one end only for item / data insertion, and one end for data removal, it is similar to the real life concept of queue or line. A queue needs **two pointers** because elements enter at one end and leave from the other:

- front → points to the element that will be removed next
- rear → points to the last element added (or the next insertion position)

• Core Operations

Operation	Description	Alternative Names
enqueue()	Add element to the **rear/back**	`add()`, `offer()`
dequeue()	Remove and return element from the **front**	`remove()`, `poll()`
front()	View front element without removing	`peek()`, `element()`
isEmpty()	Check if queue has no elements	`isempty()`
isFull()	Check if queue has reached capacity	(for bounded queues)



For code checkout this link:

https://github.com/Sandesh775/DSA_BCA_Course/blob/main/Data%20Structures/Queue/Queue_Implementation_Using_Array/Implementing_Queue.java

Comparison of All Three Approaches

Aspect	Approach 1	Approach 2	Approach 3
front start	`0`	`0`	`-1`
rear start	`-1`	`0`	`-1`
Empty condition	`size == 0`	`size == 0` or `front == rear`	`front == -1`
First element	`rear++` then insert	insert then `rear++`	`front=0`, `rear++`
Last element	`front++`	`front++`	Both reset to `-1`
Size variable?	Usually yes	Usually yes	Can track or compute

Circular Queue

A circular queue is a linear data structure that improves upon a standard queue by connecting the last position back to the first, forming a ring structure. It operates on the FIFO principle, but allows re-using empty spaces created by dequeuing, solving the memory wastage issue of linear queues.

Core part remember is use this modulo operator:

rear = (rear+1)%arr.length; // rear = (front + Current_size) % capacity;

front = (front+1)%arr.length;

isEmpty	Check if no elements	size == 0 or front == rear (with wasted slot)
isFull	Check if queue is full	size == capacity or (rear + 1) % capacity == front

For code check this link:
https://github.com/Sandesh775/DSA_BCA_Course/blob/main/Data%20Structures/Queue/Queue_Implementation_Using_Array/Circular_Queue.java

Deque : Double ended queue

A **Deque** (pronounced "deck") is a **Double-Ended Queue** that allows insertion and deletion from **both ends** — front and rear.

Deque Operations

Operation	Description	Alternative Names
insertFront()	Add element at front	`addFirst()`, `offerFirst()`
insertRear()	Add element at rear	`addLast()`, `offerLast()`
deleteFront()	Remove from front	`removeFirst()`, `pollFirst()`
deleteRear()	Remove from rear	`removeLast()`, `pollLast()`
getFront()	View front element	`peekFirst()`
getRear()	View rear element	`peekLast()`

Deque as a Combination

Deque can behave like **both** a stack and a queue:

If you use...	It behaves like...
`insertRear()` + `deleteFront()`	Queue (FIFO)
`insertRear()` + `deleteRear()`	Stack (LIFO)
`insertFront()` + `deleteFront()`	Stack (LIFO from front)
`insertFront()` + `deleteRear()`	Reversed queue

For code:
https://github.com/Sandesh775/DSA_BCA_Course/blob/main/Data%20Structures/Queue/Queue_Implementation_Using_Array/Deque_Double_ended_Queue.java